

Introduction-

This report aims to explore convergence time and model accuracy with different optimization/learning rate options in TensorFlow using both shallow and deep networks on the toy dataset from the Higham and Higham paper and also a more complex dataset from Kaggle. This report in particular covers Stochastic Gradient Descent, Adam, Adagrad, Adadelata, Nadam, Adamax, and RMSprop. The experiments aimed to understand how model architecture and optimization algorithms influence learning dynamics, stability, and final performance.

All experiments were implemented in TensorFlow/Keras using the same random seed for reproducibility. Early stopping monitored validation loss to prevent overfitting, and all models were trained with standardized input features and consistent initialization.

Toy Dataset-

Dataset and Setup-

The toy dataset consisted of ten manually defined two-dimensional points divided evenly into two classes (five positive and five negative). This small, linearly separable dataset allowed for rapid testing of optimizer behavior under controlled conditions. Both the shallow and deep networks were trained on the entire dataset (no cross-validation) using categorical cross-entropy loss and a batch size equal to the number of samples.

Two architectures were used:

Shallow model: 1 wide hidden layer consisting of 64 units with a ReLu activation followed by a 2-unit softmax output

Deep Model: 5 hidden layers (64 -> 32 -> 16 -> 8 -> 4) with ReLu activations followed by a 2-unit softmax output.

Each optimizer was tested at learning rates of $1e-3$ and $1e-2$ (and for Adadelata, 1.0), with training capped at 200 epochs and early stopping enabled to detect convergence plateaus.

Results-

The results showed that most optimizers achieved perfect accuracy (100%) on the toy dataset when trained on all ten samples. Specifically, Adam, Adamax, Nadam, RMSprop, and SGD all reached 1.0 accuracy across both the shallow and deep architectures at learning rates of either $1e-3$ or $1e-2$.

The number of epochs to convergence varied slightly (typically between 137 and 200), but the performance consistency across these optimizers demonstrates their robustness on small, well-behaved data. In contrast, Adagrad occasionally plateaued at 0.9 accuracy due to its cumulative learning-rate decay, which causes updates to shrink over time. Adadelata, when used with very low learning rates ($1e-3$ – $1e-2$), failed to converge beyond 0.4–0.6 accuracy, confirming that its effective default rate of 1.0 is critical for progress.

Overall, all adaptive optimizers successfully overfit the dataset (as expected for such a small sample). The consistent perfect accuracy across optimizers confirmed that the implementation worked correctly and that the training setup was stable. The toy dataset thus served as a useful diagnostic tool: it revealed how different optimizers behave in extreme small-data conditions, provided insight into convergence sensitivity, and confirmed that the training framework was stable before applying it to larger-scale experiments.

Kaggle Dataset-

Dataset and setup-

The second phase of experimentation used the Wisconsin Breast Cancer (Diagnostic) dataset, which contains 569 samples and 30 real-valued standardized features describing characteristics of cell nuclei. Each sample is labeled as benign or malignant. This time 30% of the dataset was held out as a test set in order to test the generalization of the model on data that it has not seen.

Two similar architectures were evaluated:

Shallow model: 1 wide hidden layer consisting of 64 units with a ReLu activation followed by a 1 unit sigmoid output

Deep Model: 5 hidden layers (64 -> 32 -> 16 -> 8 -> 4) with ReLu activations followed by a 1 unit sigmoid output.

Both models used binary cross-entropy loss and identical preprocessing. Early stopping monitored validation loss with a patience of 40 epochs to prevent overfitting. The maximum epoch limit was set to 400.

Optimizer Configuration-

Seven optimizers were tested (Adam, Adamax, Nadam, RMSprop, SGD, Adagrad, and Adadelata), and each initialized with sensible default learning rates based on prior experimentation:

- Adam-family (Adam, Adamax, Nadam) and RMSprop: $lr = 1e-3$
- Adagrad: $lr = 5e-2$
- Adadelata: $lr = 1.0$
- SGD (with momentum = 0.9, Nesterov enabled): $lr = 5e-2$

These defaults were informed by the toy dataset experiments, which showed that excessively small learning rates cause stagnation in Adadelata and Adagrad, while $1e-3$ - $1e-2$ works consistently for Adam-type optimizers.

Results-

The overall test accuracies ranged from 95.9% to 99.4%, with most optimizers performing very well. The best performance was achieved by the deep network with Adamax ($lr = 0.001$), reaching a 99.4% test accuracy. The Adam, Adagrad, and Adadelata optimizers followed closely at approximately 98.8%, while SGD and RMSprop achieved between 97–98%.

Training curves showed that the deep network typically required 150–400 epochs to converge, whereas the shallow network reached similar performance in 60–120 epochs. This pattern indicates that additional depth increases training time but yields only a marginal accuracy improvement (~0.5–1%).

These findings parallel the toy dataset: once the optimizer is stable and the learning rate is appropriate, both shallow and deep models can fit the data effectively. Adaptive optimizers proved to be more robust across architectures, while SGD required careful tuning but remained competitive when configured correctly.

Discussion and Conclusion-

The combined experiments across toy and real datasets demonstrate a consistent pattern:

1. Adaptive optimizers (Adam, Adamax, Adagrad, Adadelat) provide smooth and reliable convergence across model depths and dataset complexities.
2. Model depth increases capacity but not necessarily performance, particularly on simple or well-structured datasets.
3. Shallow networks can be highly effective, offering near-optimal accuracy with faster convergence and lower computational cost.

The toy dataset illustrated fundamental optimization behavior and verified model correctness, while the breast cancer dataset confirmed these findings on a larger, real-world problem. The deep Adamax model achieved the best result (99.4%), but the shallow Adam model achieved nearly the same accuracy (98.8%) with fewer epochs.

In practical terms, for at least a dataset similar to the chosen one , a single-hidden-layer network with an adaptive optimizer like Adam or Adamax provides an ideal balance of performance and efficiency. Depth may yield marginal gains but at the expense of training time and parameter complexity.

Together, these experiments reinforce the importance of optimizer selection over architectural depth for moderate-sized, low-noise datasets and illustrate how even simple neural networks can achieve near-perfect results when paired with appropriate optimization strategies.